

F24-019-D-DataTails

Project Team

Shitba Kashif	21I-2676
Fasih Ur Rehman	21I-1705
Maryam Noor Fatima	21I-2656

Session 2021-2025

Supervised by

Dr. Arshad Islam



Department of Data Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Sprint Plan	3
1.3.1	Sprint 1 (September – October)	3
1.3.2	Sprint 2 (November – December)	4
1.3.3	Sprint 3 (January – February)	4
1.3.4	Sprint 4 (March – April)	5
1.4	User Classes and Characteristics	6
2	Project Requirements	7
2.1	Use-case Representation	7
2.1.1	Use Case Description	7
2.1.2	Use Case Representation	8
2.2	Functional Requirements	9
2.2.1	Module 1: Data Scraping and Ingestion	9
2.2.2	Module 2: Data Transformation and Knowledge Graph Construction	11
2.2.3	Module 3: Advanced Querying and Insights Generation	11
2.2.4	Module 4: System Integration and Testing	11
2.3	Non-Functional Requirements	12
2.3.1	Reliability	12
2.3.2	Usability	13
2.3.3	Performance	13
2.3.4	Security	13
2.4	Domain Model	13
2.4.1	Entities and Relationships	14
3	System Overview	17
3.1	Architectural Design	18
3.1.1	Relationships Between Modules	19
3.1.2	Architectural Design Diagram	19
3.2	Design Models	20
3.2.1	JSON Schema Diagram	20

3.2.2	Design Models for Procedural Approach	22
3.2.3	Activity Diagram	23
3.2.4	Data Flow Diagram	25
3.2.5	System-Level Sequence Diagram	27
3.2.5.1	Explanation of the Sequence Flow	27
3.2.5.2	Key Components of the Sequence Diagram	28
3.2.6	State Transition Diagram	28
3.2.6.1	Explanation of the State Flow	29
3.2.6.2	Key Components of the State Transition Diagram	30
3.3	Data Design	30
References		37

List of Figures

1.1	DataTails Workflow Architecture	2
1.2	Detailed Version of Sprint 1	3
1.3	Explanation of Sprint 2	4
2.1	Use Case Diagram for DataTails	8
2.2	Number of Subscribers per Subreddit	10
2.3	Scores by Subreddit Based on Upvotes and Subscribers	10
2.4	Domain Model for the DataTails System	14
3.1	Architectural Design Diagram for DataTails	20
3.2	JSON Schema Diagram	21
3.3	Activity Diagram for DataTails	32
3.4	Level 2 Data Flow Diagram for DataTails	33
3.5	Level 0 DataFlow Diagram	34
3.6	Level 1 DataFlow Diagram	35
3.7	System-Level Sequence Diagram for DataTails	35
3.8	State Transition Diagram for DataTails	35
3.9	MongoDB JSON Schema Diagram for Posts and Comments	36

List of Tables

Chapter 1

Introduction

DataTails discusses the design and development of a data analytics platform that transforms unstructured Reddit data into insightful perspective. By using Large Language Models (LLMs), Knowledge Graphs, and a Visualization Recommendation Engine, DataTails offers sophisticated query functionalities, natural language processing (NLP), and user friendly visual representations. This platform is designed to meet the increasing demand for accessible tools that can efficiently process large, unstructured datasets such as those from Reddit and facilitate advanced data analysis.

1.1 Problem Statement

Reddit hosts an extensive collection of user-generated content across a wide range of topics. However, deriving meaningful insights from this data poses a significant challenge due to its unstructured format. Conventional data analysis tools are not well-suited for managing the intricate, context-dependent nature of Reddit discussions, where topics can evolve rapidly and user sentiments are often complex and multifaceted. Data analysts frequently face difficulties in organizing the sheer volume of content, selecting appropriate visualizations, and interpreting large-scale community-driven conversations. Furthermore, the lack of a system capable of recommending suitable visual representations or streamlining the querying process for Reddit data impedes effective analysis.

DataTails effectively addresses these challenges by processing Reddit's unstructured data through several stages, ultimately transforming it into structured and actionable insights. The core components of this solution are depicted in Figure 1.1, which illustrates the system's workflow.

The workflow starts with web scraping and continues with continuous data gathering using Kafka streams that ingest data from Reddit and scheduled jobs(Cron Jobs), as shown in the diagram. MongoDB is used to store this unstructured data. In order to organize and

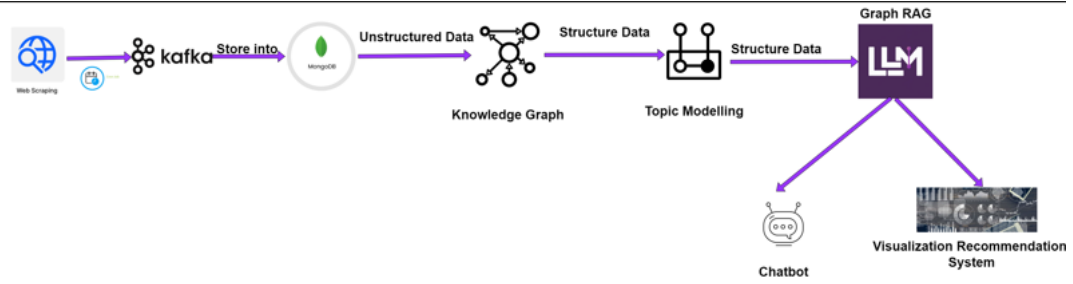


Figure 1.1: DataTails Workflow Architecture

link the data based on relationships and context, the following step is transforming this raw, unstructured data into structured representations using a knowledge graph. The system uses topic modeling to extract important themes and patterns from the material once the data has been organized[2]. In order to provide insights and offer contextually appropriate responses based on user inquiries, a RAG-based Large Language Model (LLM) is employed. Furthermore, to show the data more effectively and improve user comprehension, a Visual Recommendation System makes recommendations for the best possible representations.

DataTails' architecture provides a comprehensive solution to the challenges posed by unstructured Reddit data, enabling users to effectively analyze large-scale datasets and gain useful insights.[1].

1.2 Scope

DataTails intends to create a powerful analytics platform developed exclusively for large-scale, unstructured Reddit datasets. The platform enables users to make natural language inquiries, relying on integrated Large Language Models (LLMs) to give contextually accurate insights that are reinforced by Knowledge Graphs. One of its most notable features is the Automatic Visualization Recommendation Engine, which recommends ideal renderings based on the examined data. The platform includes crucial functionalities for data analysts, organizations, and researchers, such as sentiment analysis, trend identification, and subject modeling.

DataTails will be accessible via a web application, with premium users benefiting from advanced features. Premium users will not only access enhanced capabilities but also gain in-time data insights through live scraping of Reddit at designated intervals. Initially, the platform's scope will focus on Reddit data, with the potential for future expansion to other data sources.

1.3 Sprint Plan

The proposed project will be completed over four sprints/iterations. Each sprint focuses on specific modules and their development to ensure the platform's functionality evolves in a structured manner.

1.3.1 Sprint 1 (September – October)

Focus: Core Data Consumption and Foundation

- **Web Scraping Module**
 1. Real-time extraction of data from Reddit.
 2. Scrapped datasets are constantly updated and maintained.
- **Data Ingestion & Storage Module (Kafka + MongoDB)**
 1. Kafka enables in-time data ingestion.
 2. Efficient unstructured data storage using MongoDB.
- **Frontend Development Module (Part 1: Basic Configuration)**
 1. Simple and user-friendly interface.
 2. Initial interactive dashboard (basic querying capabilities).
- **Integration Module (Early Version)**
 1. Web scraping and data input are integrated with MongoDB.

Figure 1.2 shows detailed form of Sprint 1.

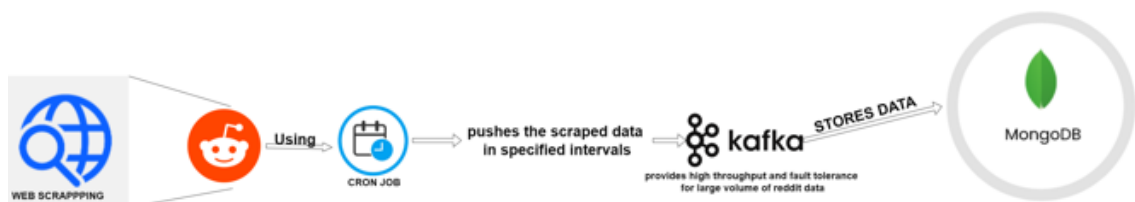


Figure 1.2: Detailed Version of Sprint 1

1.3.2 Sprint 2 (November – December)

Focus: Data Structuring and Initial Logic Implementation

- **Knowledge Graph Module**
 1. Conversion of unstructured data to a structured format.
 2. Contextual ties added to data to improve querying.
- **Topic Modeling Module**
 1. Automated identification of key topics.
 2. Classifying discussions according to trends.
- **Frontend Development Module (Part 2: Enhanced Interface)**
 1. Add advanced query functionalities.
 2. Highlight themes and trends derived from data.
- **Testing Module (Phase 1)**
 1. Begin unit testing for the web scraping, knowledge graph, and topic modeling modules.

Illustrated in Figure 1.3 is the logic of Sprint 2 for better understanding.

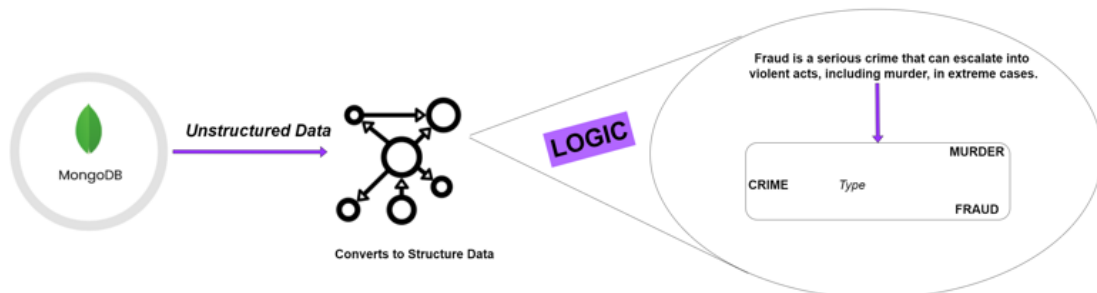


Figure 1.3: Explanation of Sprint 2

1.3.3 Sprint 3 (January – February)

Focus: Advanced Querying and Contextual Responses

- **Graph-RAG Module**
 1. Integration of knowledge graphs with LLMs.

2. Generation of context-aware, accurate responses.

- **Chatbot Module**

1. Natural language querying interface.
2. Context-aware responses based on LLMs and knowledge graphs.

- **Visualization Recommendation System Module**

1. Automated visualization suggestions.
2. Support for several kinds and formats of charts.

- **Backend Integration & API Gateway Module (Part 1)**

1. Chatbot and visualization modules integrated with backend services.
2. Fundamental management of APIs for safe communication.

1.3.4 Sprint 4 (March – April)

Focus: Final Integration, Testing, and User Interface

- **Testing Module (Phase 2)**

1. Complete testing includes performance, integration, and unit testing for every component.
2. Verify that the system's benchmarks are fulfilled.

- **Integration Module (Final Phase)**

1. Every system module is integrated seamlessly.
2. Complete workflow administration and data processing.

- **Frontend Development Module (Final Phase)**

1. Complete features for Reddit data analysis, visualization, and querying.
2. Ensure user-friendly chatbot and visualization interaction by refining the user interface.

- **Backend Integration & API Gateway Module (Part 2)**

1. Complete API management.
2. User authentication and role-based access control (normal vs. premium).
3. Feature management based on user tiers.

1.4 User Classes and Characteristics

DataTails will be used by different user classes, including Premium and Non-premium users. Premium users have access to advanced features like visualization recommendations, while Non-premium users have access to basic functionalities. Additionally, system users and data source components (such as Reddit and Kafka) interact to facilitate data scraping, processing, and streaming. Below Table shows details:

User Case	Description
Developer	A Developer in DataTails is responsible for configuring and maintaining various system integrations and data management tasks. This includes setting parameters for data extraction, cleaning, and curation, defining criteria for filtering and preprocessing Reddit data (e.g., identifying specific subreddits or content types), and ensuring compliance with platform standards and policies. Additionally, Developers manage the integration with Language Models (LLMs), which enhances data analysis capabilities. They handle natural language query support, tune LLM responses, and adjust model parameters to improve accuracy and relevance in response to user interactions.
End User	End Users are external stakeholders or clients who access processed insights and reports generated by DataTails. They primarily view key performance indicators (KPIs) and trends without access to raw data.
Premium User	A Premium User has subscribed to DataTails for advanced features, such as receiving visualization recommendations, access to detailed knowledge graphs, and personalized topic analysis. These users are typically data analysts or business professionals who require deeper insights from the data processed by DataTails.
Non-premium User	A Non-premium User can access basic features of DataTails, such as submitting queries and selecting topics for data scraping and processing. Non-premium users may include casual users or those interested in exploring data insights without needing advanced visualizations or personalized recommendations.
System User	The System User interacts with DataTails for the overall maintenance, monitoring, and configuration of the data processing pipeline. System Users are typically administrators responsible for setting up data scraping, managing Kafka integration, and ensuring the reliability of the data streams and knowledge graph generation.
Data Source Component	DataTails uses data sources, such as Reddit, for scraping unstructured data on selected topics. These data sources are integral to the system and provide the raw data needed for analysis. Data scraping is conducted based on user-selected topics, and this data is further processed to generate meaningful insights.
Kafka Integration	Kafka acts as the streaming component, enabling processed data to be streamed to users in real time. This integration ensures scalability and availability of data streaming and allows users to access up-to-date information. Kafka integration is managed by system users or developers.

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of DataTails.

2.1 Use-case Representation

The requirement gathering technique applied in the DataTails project includes the use case model, as it is an effective approach for interactive end-user applications involving data analysis. The use case diagram below illustrates the interactions between different users (both premium and non-premium) and the system components to fulfill their data analysis requirements.

2.1.1 Use Case Description

In the DataTails use case, both premium and non-premium users interact with the system to perform data analysis tasks. The interactions are represented in the use case diagram below:

- **User (Premium and Non-premium) Interactions:**
 - The user starts by **submitting a query** to the system.
 - Premium users have the additional feature of **receiving visualization recommendations**, which extends the basic query submission functionality.
 - All users are required to **select a topic** for which they want to gather and analyze data.
- **System Operations:**

- The system then proceeds to **scrape data** from a specified source, which is Reddit in this use case.
- After scraping, the system **processes the unstructured data** to clean and transform it for analysis.
- The processed data is used to **generate a knowledge graph**, providing a visual representation of insights derived from the data.
- Finally, the system **streams the processed data via Kafka**, enabling in-time access for users or other integrated systems.

2.1.2 Use Case Representation

The use case diagram [2.1](#) below visually represents these interactions:

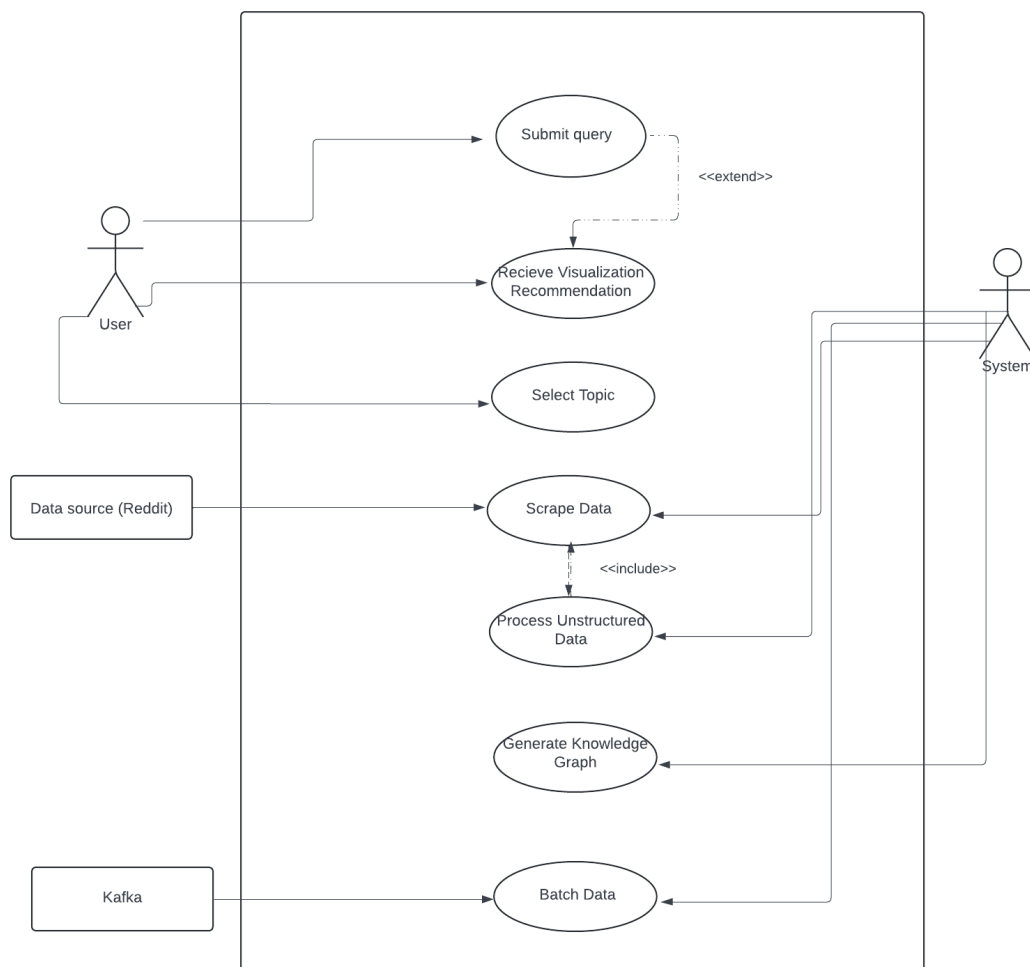


Figure 2.1: Use Case Diagram for DataTails

The use case diagram illustrates the key actors and interactions:

- **User (Premium and Non-premium):** Initiates the data analysis process.
- **System:** Handles data scraping, processing, knowledge graph generation, and streaming.
- **Data Source (Reddit):** Acts as a source for unstructured data.
- **Kafka:** Manages in-time data streaming.

Each oval represents a distinct functionality, from initiating a query to the final streaming of the data, showcasing how DataTails handles user requests and processes the data for visualization and in-time access.

2.2 Functional Requirements

This section describes the functional requirements of the DataTails platform, organized by system module and associated features.

2.2.1 Module 1: Data Scraping and Ingestion

This module is responsible for gathering and storing Reddit data. Refer to figure 1.2 for detailed view.

1. **Reddit Scraping** - The system shall scrape Reddit using PRAW, Selenium, or BeautifulSoup to extract posts, comments, user data, and metadata from designated subreddits.
2. **Logic for Choosing Subreddits** - To effectively choose subreddits to collect data from, we selected subreddits with high context, high volume, and high traffic. We gathered a list of subreddits from various topics, including: Pakistan, Technology, News, Politics, Travel, Relationships, etc.

Once these subreddits were listed, we employed a method to determine which ones to choose for data collection. Each subreddit has a count of subscribers (followers) and each post has a number of upvotes. To evaluate suitability, we calculated the average number of upvotes from 2000 posts in each subreddit, along with the maximum and minimum number of upvotes. Additionally, we determined subreddits with maximum and minimum subscribers.

To normalize these values, min-max normalization was applied to both upvotes and subscriber counts for each subreddit. With the normalized data, we calculated a

score using the formula:

$$\text{score} = w_1 \times \text{normalized_upvotes} + w_2 \times \text{normalized_followers}$$

The score ranged from 0 to 1, with values closer to 1 indicating a better subreddit in terms of volume and traffic. Both subscribers and upvotes were used to reduce bias and ensure an effective selection of subreddits.

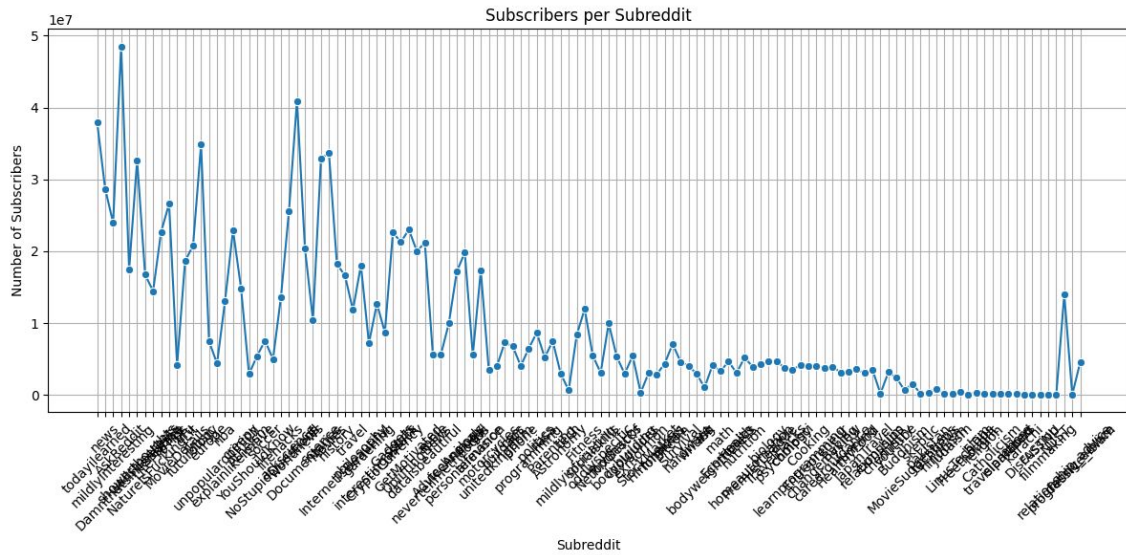


Figure 2.2: Number of Subscribers per Subreddit

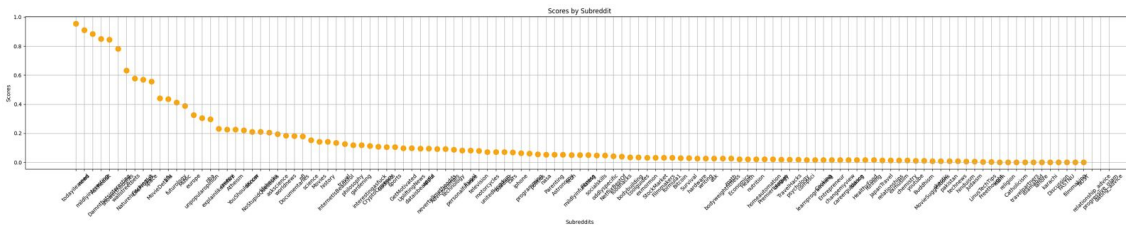


Figure 2.3: Scores by Subreddit Based on Upvotes and Subscribers

3. **Continuous Data Gathering** - The system shall use cron jobs to schedule scraping tasks at predefined intervals to ensure up-to-date data collection.
4. **Data Ingestion** - Kafka shall be used for in-time data ingestion from the scraping module, ensuring seamless data flow into MongoDB.
5. **Storage and User Differentiation** - MongoDB shall store data with differentiation between premium users (who have access to live scraping and in-time data) and non-premium users (who access archived data).

2.2.2 Module 2: Data Transformation and Knowledge Graph Construction

This module focuses on transforming unstructured Reddit data into a structured format using Knowledge Graph (KG). Refer to figure [1.3](#) for detailed view.

1. **Data Transformation** - The system shall convert raw, unstructured Reddit data (e.g., post titles, descriptions, comments) into structured data by employing techniques such as topic modeling and entity extraction.
2. **Ontology Integration** - The system shall search for and integrate suitable ontologies (such as SIOC, DCMI, FOAF) for structuring the KG. If existing ontologies do not fit well, the system shall allow for custom ontology creation or extensions to ensure compatibility with Reddit data.
3. **Monthly KG Reconstruction** - The system shall perform periodic Knowledge Graph reconstruction to update the structure with new data, ensuring that the KG reflects the latest Reddit discussions and relationships.

2.2.3 Module 3: Advanced Querying and Insights Generation

This module deals with providing users insights from the structured data.

1. **RAG-based LLM** - The system shall integrate Retrieval-Augmented Generation (RAG) based Large Language Models (LLMs) to respond to user queries by extracting relevant knowledge from the KG.
2. **Query Processing** - The system shall accept user queries via a natural language interface, process the queries, and extract relevant information using KG and RAG-based LLMs.
3. **Visualization Recommendation** - The system shall recommend appropriate visualizations for displaying the queried data based on the user's input and context, offering various chart formats.

2.2.4 Module 4: System Integration and Testing

This module is responsible for the integration and testing of all components of DataTails, with distinct functionalities for premium and non-premium users.

1. **API Integration** - The system will combine a variety of APIs, including data scraping, topic modeling, and KG-based querying APIs, to ensure easy communication between modules.
2. **LLM Integration** - The system will connect RAG-based Large Language Models (LLMs) with the Knowledge Graph (KG) to handle user queries and offer accurate insights.
3. **Visualization System Integration** -The system will include a visualization recommendation engine to offer users appropriate and context-aware visuals depending on their query.
4. **User Access Levels Integration:**
 - **Premium Users** - Premium users gain access to premium features including expanded query limits, comprehensive visualizations, and personalized topic modeling insights.
 - **Non-Premium Users** - Non-premium users will have access to basic functionality, including a limited amount of queries, fewer visualizations, and restricted access to specific KG data.
5. **Testing** - The system will be thoroughly tested to ensure that all attributes, including premium and non-premium access controls, Knowledge Graph , LLM integration, and visualization recommendations, function properly.
6. **Continuous Integration and Monitoring** - The system shall incorporate a continuous integration pipeline to test new updates and monitor system performance, ensuring that premium and non-premium user experiences are distinct and function as expected.

2.3 Non-Functional Requirements

This section outlines the quality attributes that the DataTails platform must adhere to, ensuring it meets user expectations in terms of performance, reliability, security, and usability.

2.3.1 Reliability

- REL-1: The system shall have a mean time between failures (MTBF) of 99.95%, ensuring high availability for premium users who rely on in-time insights.

- REL-2: The system shall detect scraping or ingestion failures and retry operations within 5 minutes of failure detection.

2.3.2 Usability

- USE-1: The platform shall allow a user to perform a text-based search query and receive insights with no more than 3 interactions (query input, clarification, result).
- USE-2: The system's interface shall present recommended visualizations in a drag-and-drop format, allowing users to further refine their selections.

2.3.3 Performance

- PER-1: 95% of responses to user queries shall be generated within 3 seconds, with data loading times for visualizations being under 2 seconds for non-premium users and under 1 second for premium users.
- PER-2: Kafka shall ingest and process incoming data streams from Reddit within 2 seconds of receipt, ensuring in-time data flow.

2.3.4 Security

- SEC-1: The system shall prevent unauthorized access to premium user features by implementing role-based access control (RBAC) and OAuth 2.0 for authentication.
- SEC-2: The system's backend shall be secured using HTTPS encryption to protect user data during transmission.
- SEC-3: The system shall log all user interactions for auditing and flagging suspicious activity, with attempts to breach the system being reported in in-time.

2.4 Domain Model

The Domain Model in Figure 2.4 provides a high-level representation of the main entities, their attributes, and relationships in the system. Each entity plays a specific role in capturing and processing user queries and generating visualizations based on the underlying data. This diagram helps illustrate the relationships and dependencies between different components involved in the data analysis process.

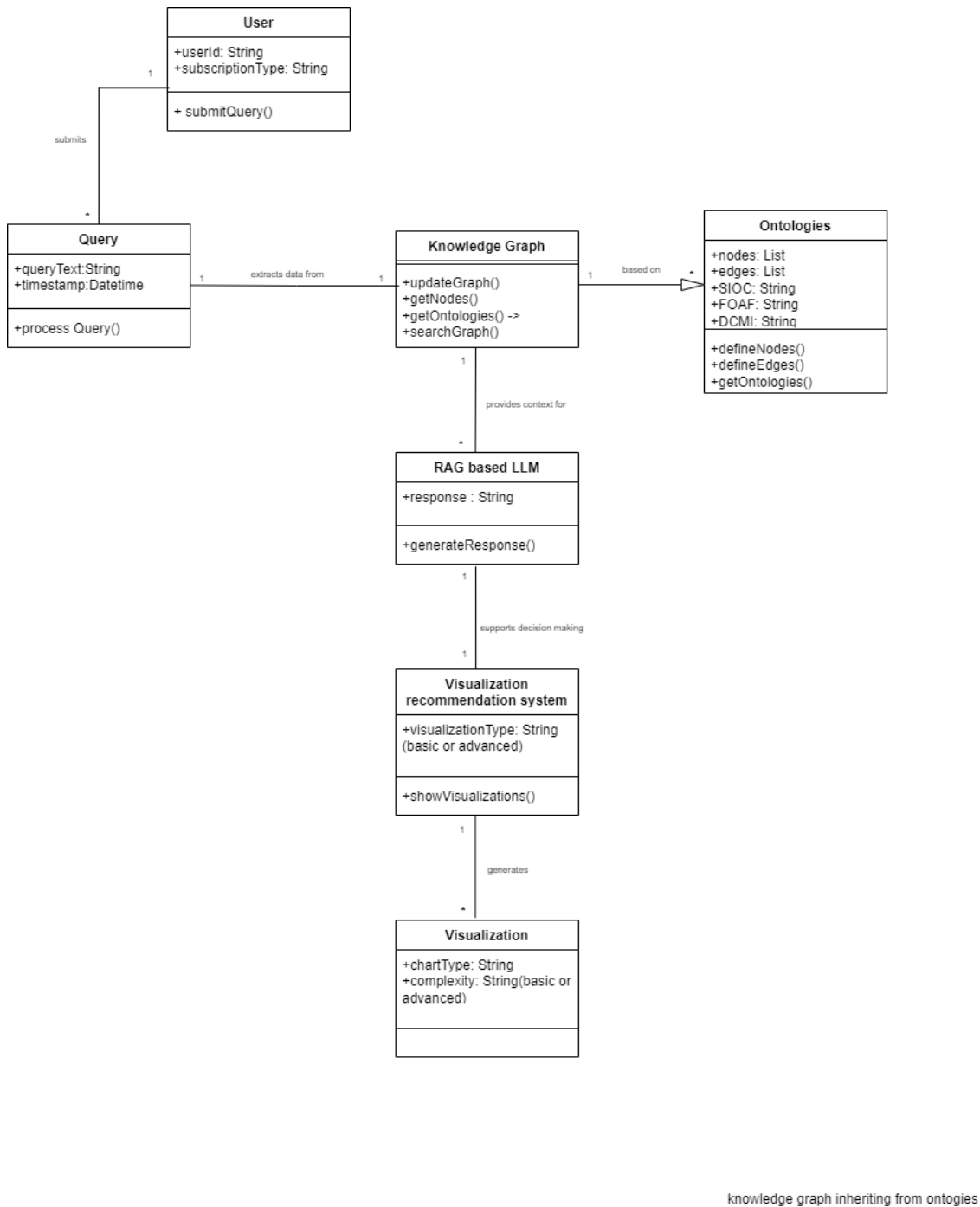


Figure 2.4: Domain Model for the DataTails System

2.4.1 Entities and Relationships

- **User:** Represents the individuals using the system. The attributes include `userId` and `subscriptionType`. The method `submitQuery()` enables a user to submit a query to the system.

- **Query:** This entity captures the user query. It contains `queryText` and `timestamp`, representing the text of the query and when it was submitted. The relationship between **User** and **Query** is represented as "submits", meaning a user can submit multiple queries.
- **Knowledge Graph:** The **Query** entity interacts with the **Knowledge Graph**, which provides extracted information from the data. The **Knowledge Graph** has methods like `updateGraph()`, `getNodes()`, `getOntologies()`, and `searchGraph()`. The Knowledge Graph provides the necessary context for further processing.
- **Ontologies:** The **Knowledge Graph** is based on **Ontologies**, inheriting some core definitions and data structures. Attributes include `nodes`, `edges`, and specific ontologies like SIOC, FOAF, and DCMI. The relationship is defined as "based on", representing the dependency of the **Knowledge Graph** on **Ontologies**.
- **RAG Based LLM:** This component uses the contextual information from the **Knowledge Graph** to generate relevant responses. It supports decision-making for further data processing, with methods such as `generateResponse()`.
- **Visualization Recommendation System:** This entity processes the context from the **RAG Based LLM** to determine suitable visualizations. The attribute `visualizationType` defines whether a visualization is basic or advanced, and the method `showVisualizations()` generates recommendations.
- **Visualization:** The final output is represented as a **Visualization**, with attributes like `chartType` and `complexity`. This entity captures the visual representation that will be displayed to the user. The relationship "generates" indicates that the **Visualization Recommendation System** generates the **Visualization**.

Chapter 3

System Overview

DataTails is a data analysis and visualization platform designed to help users gather insights from unstructured data, specifically from sources like Reddit. The system provides both premium and non-premium users with the ability to submit queries, select topics of interest, and receive valuable insights through processed data and visualizations.

The primary functionality of DataTails includes:

- **Query Submission:** Users can submit specific queries to analyze topics of their choice.
- **Data Scraping:** Data is scraped from Reddit based on the selected topic, providing the raw information needed for analysis.
- **Data Processing:** The scraped data is processed to transform unstructured content into a form suitable for analysis, including cleaning, filtering, and structuring.
- **Knowledge Graph Generation:** The system constructs knowledge graphs to assist users visualize relationships within the data and gain a better grasp of the information.
- **In-time Data Streaming:** Kafka is used to stream processed data, ensuring that users have access to the most up-to-date information.
- **Visualization Recommendations (for Premium Users):** Premium users receive recommendations on the finest data visualization strategies.

DataTails works within a broad ecosystem that includes:

- **Users:** Both premium and non-premium users interact with the system, however premium users have access to more advanced functions.

- **Data Source:** Reddit is utilized as a data source for extracting important information from user-selected subjects.
- **Kafka Integration:** Kafka allows for the streaming of processed data, ensuring real-time access for users and integrating with other systems.
- **System Components:** The system has components for data scraping, processing, knowledge graph building, and streaming.

DataTails is intended to provide an interactive and informative experience for anyone seeking to extract meaning from massive amounts of unstructured data. The system's architecture makes use of modern data processing techniques and in-time streaming capabilities to ensure that users obtain current and correct information for decision-making.

3.1 Architectural Design

DataTails' architectural design takes a modular approach, integrating multiple technologies to deliver full data analysis and visualization capabilities. The architecture is organized into distinct modules, each responsible for a certain task, enabling the system's scalability, maintainability, and extensibility. The following is a high-level summary of how the system's modules interact to provide the desired functionality.

The architecture of DataTails includes the following major components:

- **Frontend (React.js):** The frontend is built with React.js and serves as the primary interface for users. It enables both premium and non-paid users to submit searches, analyze visualizations, and engage with data. The frontend makes use of visualization libraries like **Plotly.js**, **d3.js**, and **Chart.js** to create interactive and visually appealing data representations.
- **Backend (Flask):** The backend uses Flask to handle user queries, manage data flow between the frontend and other system components, and link with other data sources like Reddit via the **Reddit API**. It regulates communication between the system's many modules.
- **Web Scraper:** The web scraper is in charge of collecting data from Reddit via the Reddit API. It collects and stores unstructured data in **MongoDB** for later processing.
- **MongoDB:** MongoDB serves as a fundamental database for storing the scraped data. It manages unstructured data in a dependable and scalable manner, allowing other modules to access it more easily.

- **Topic Modelling Module:** This module extracts key themes from MongoDB data and organizes the information sensibly. The extracted topics are then utilized to construct a **Knowledge Graph** and provide inputs to the **RAG-based LLM**.
- **Knowledge Graph and RAG-based LLM:** The **Knowledge Graph** visualizes the relationships between various subjects and entities retrieved from the data. The RAG-based LLM gives context-aware replies to user questions based on processed data, boosting information quality.
- **Visual Recommendation System:** This module recommends appropriate visualizations to users based on insights gained from the knowledge graph and the RAG-based LLM, so improving the overall user experience and increasing access to data insights.
- **Kafka Stream and Cron Job Scheduler:** The **Kafka Stream** manages in time data streaming, ensuring users have access to the most recent information. The **Cron Job Scheduler** manages data scraping processes and keeps the data up to date without operator intervention.

3.1.1 Relationships Between Modules

The modules are interconnected to create an efficient data analysis pipeline:

- **Frontend (React.js)** interacts with the **Backend (Flask)** to handle user requests and display visualizations using **Plotly.js**, **d3.js**, and **Chart.js**.
- **Backend** uses the **Reddit API** to direct the **Web Scraper** to collect data, which is stored in **MongoDB**.
- **Topic Modelling Module** processes the stored data to derive topics and creates a **Knowledge Graph** and interacts with the **RAG-based LLM** for advanced analysis.
- **Visual Recommendation System** uses the generated insights to recommend appropriate visualizations.
- **Kafka Stream** ensures in-time data access, while the **Cron Job Scheduler** automates the data scraping process.

3.1.2 Architectural Design Diagram

The following diagram provides a visual representation of the architectural design of **DataTails**, highlighting the major subsystems and their connections:

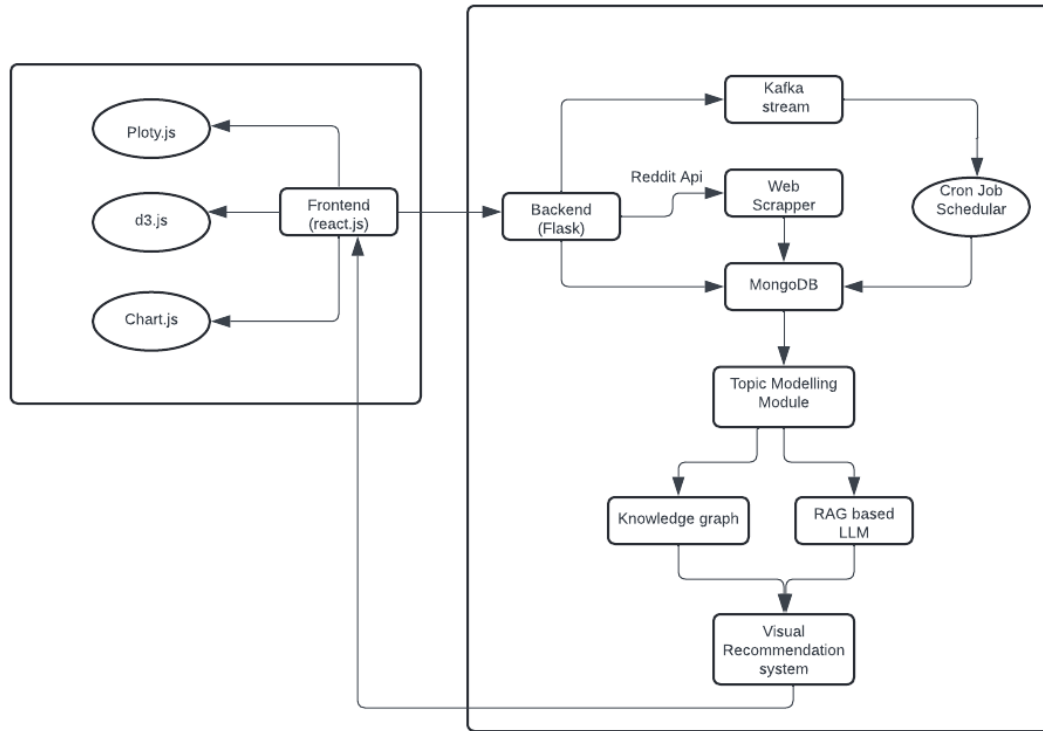


Figure 3.1: Architectural Design Diagram for DataTails

This architectural diagram provides a box-and-line representation of the system, showing the integration of the frontend, backend, data processing, and visualization components, along with external systems like Reddit and Kafka. The design follows a layered and modular approach to ensure clean separation of responsibilities and ease of maintenance.

3.2 Design Models

3.2.1 JSON Schema Diagram

The JSON schema diagram, shown in Figure 3.2, represents the structure of the key entities in the Reddit data processing platform, including their attributes and relationships. This diagram provides a clear view of how **Post**, **Comment**, and **User** entities are defined and how they interact with each other. Each entity is denoted by a class-like structure that includes a list of its attributes, and relationships are represented by labeled arrows that indicate the direction and type of the interaction.

Entities:

- **Post**: Represents a Reddit post with several attributes:

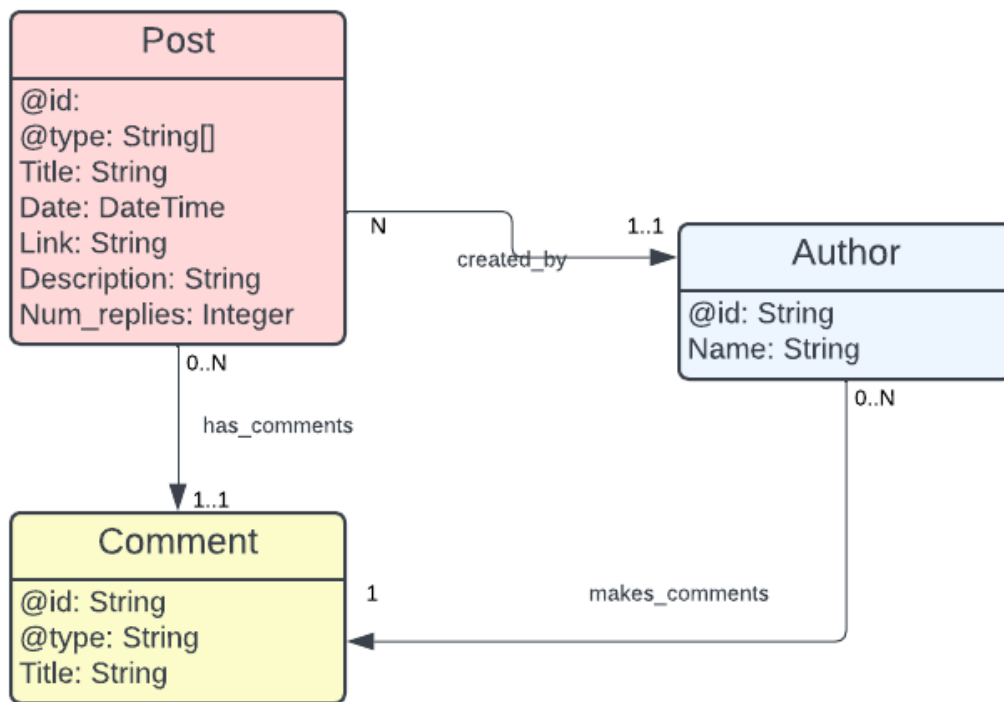


Figure 3.2: JSON Schema Diagram

- **@id:** A distinct identification for every post.
- **@type:** A list indicating the type of the post.
- **Title:** This is the title of the post.
- **Date:** The date and time of post creation.
- **Link:** A URL link for the post.
- **Description:** Brief summarization the post's content.
- **Num_replies:** The number of comments or responses to the post.
- **Comment:** Represents a comment added to a post. Characteristics include:
 - **@id:** A unique identifier for every comment.
 - **@type:** The entity's type, which is often a comment.
 - **Title:** The text of the comment.
- **User:** Author of the post who creates posts and add comments:
 - **@id:** Id is unique identifier for each author.
 - **Name:** Name of author.

Relationships:

- **Post (created_by) User:** This relationship shows that a **Post** was written by a **User**. The **created_by** label reflects the connection, indicating that each **Post** is tied to exactly one **User** (1..1), whereas a **User** can generate many **Posts**.
- **Post (has_comments) Comment:** This relationship demonstrates that a **Post** may have several **Comments**. The **has_comments** label specifies that a **Post** may contain zero or many comments (0..N), but each **Comment** belongs to exactly one **Post** (1..1).
- **User (makes_comments) Comment:** This relationship indicates that a **User** may create several **Comments**. The **makes_comments** label indicates that a **User** can write many comments (0..N), but each **Comment** is authored by just one **User** (1).

Diagram Explanation:

- The **Post** entity, as seen in **ink**, has attributes that determine its properties, including title, description, and date. It is linked to the **User** entity via the **created_by** relationship.
- The **Comment** entity, displayed in **yellow**, comprises properties such as **@id** and **Title**, which represent individual comments placed on a post. The **has_comments** relationship between **Post** and **Comment** shows that one post can have several comments.
- The **User** entity, shown in **blue**, represents a person who uses the platform to create posts and comments. The **makes_comments** relationship illustrates the connection between users and the comments they generate.

This JSON schema diagram successfully depicts the data structure and essential relationships between entities, allowing for a better understanding of how the platform manages posts, users, and comments in a structured manner.

3.2.2 Design Models for Procedural Approach

This section discusses the relevant models utilized in the project using a procedural approach. The following design models have been developed to provide a comprehensive view of the data and process flow within the system:

3.2.3 Activity Diagram

The **Activity Diagram** presented in Figure 3.3 illustrates the workflow of the **DataTails** system from a user interaction perspective, distinguishing between frontend user actions and backend processes. It shows how both premium and non-premium users interact with the system to generate visualizations.

The activity diagram is divided into two swimlanes:

- **Frontend User Actions:** This section focuses on the actions initiated by the user and the steps performed by the system interface.
- **Backend Processes:** This swimlane covers the server-side processes that support the visualization and data retrieval tasks.

Flow of Actions in the Activity Diagram

1. **User Submits a Query:** The diagram begins when a user submits a query through the React.js UI. This action is represented by the **Start symbol** (black circle).
2. **Query is Submitted to React.js UI:** Once the user submits a query, the system takes the input and sends it to the user interface for further processing.
3. **Check User Subscription Details:** The next action is to determine the user's subscription status (Premium or Non-premium). This check is crucial to determine which visualizations are available.
4. **Decision Point - Is the User Premium?:** A decision node (diamond shape) is used to evaluate whether the user is a premium user or not.
 - If the user is **Non-premium**:
 - The system provides only **basic charts**. The action is represented in the **red box** labeled "Provide basic charts only."
 - The flow continues downwards to display the results with basic visualizations.
 - If the user is **Premium**:
 - The system **allows advanced visualizations**, which is shown in the **green box** labeled "Allow advanced visualizations."
 - The flow continues downwards to display results with all available visualizations.

5. **Topic Modelling:** In the right side of diagram (in backend processes), the system performs **Topic Modelling**, which takes the user's query and identifies key topics within the data.
6. **Generate Knowledge Graph Based on Topic:** Once the topic modelling is complete, the system generates a **Knowledge Graph** based on the identified topics. This helps to organize the data in a meaningful way that supports the visualization recommendations.
7. **Generate Contextual Response using RAG based LLM:** A **Retrieval-Augmented Generation (RAG) based Language Learning Model (LLM)** is used to generate a contextual response for the user query. This ensures that the response is enriched with data from the knowledge graph.
8. **Visualization Recommendation System:** The **Visualization Recommendation System** uses the contextual response provided by the RAG-based LLM to determine the appropriate visualizations for the query. The system either provides **basic** or **advanced** visualizations based on the user's subscription level.
9. **Display Results:** Finally, the system **displays the results:**
 - **Basic visualizations** for non-premium users.
 - **All available visualizations** for premium users.

The process ends with a **terminator symbol** (black circle with an outline) indicating that the task is complete.

Key Features of the Diagram

- **Swimlanes:** The swimlanes help distinguish between the **Frontend User Actions** and **Backend Processes**. The separation makes it clear which part of the system is responsible for each action.
- **Decision-Making:** The **decision diamond** ("Is the user premium?") demonstrates how the system branches into two different flows based on the subscription type.
- **Parallel Flow:** The **black horizontal bars** represent synchronization points. For example, after checking the subscription details, the flow either provides basic charts or allows advanced visualizations, and then synchronizes to proceed with displaying the results.

The activity diagram gives a high-level view of how the user's request moves through various system components to generate the required output, with a clear differentiation between user interactions and backend processes.

3.2.4 Data Flow Diagram

The **Data Flow Diagram** presented in Figure 3.4 illustrates how data moves through the **DataTails** system, from initial data collection to the final interaction with the user. It provides a clear depiction of the stages of processing, the transformation of unstructured data into meaningful information, and how it is ultimately used to generate insights and visualizations.

The **Context Diagram** in Figure 3.5 demonstrates the high-level interaction between **DataTails** and external entities such as Reddit, users, and LLM services. This diagram captures the system's boundaries and the flow of information in and out of the system, showcasing how data is sourced, processed, and utilized.

The key interactions depicted in the context diagram include:

- **Data Source (Reddit):** Reddit acts as the primary source of raw data, including posts, comments, and user information, that is extracted and analyzed by **DataTails**.
- **User Interaction:** Users interact with the **DataTails** system by submitting queries, viewing analytical results, and accessing data visualizations.
- **LLM Services:** DataTails interacts with external **Language Model (LLM) services** to enhance query responses, provide semantic search capabilities, and enrich data processing results for improved visualization recommendations.

Here are the primary **Functional Components 3.6 DataTails**, which expand on the essential modules and their data flows.

Data Ingestion: APIs or scraping techniques are used to get raw data from Reddit, such as posts, comments, user profiles, and other relevant metadata.

Data Processing & Storage: The ingested data is cleaned, preprocessed, and stored in a structured format, such as **MongoDB** and **Knowledge Graphs**, making it ready for analysis.

Analysis & Insights Generation: The processed data is analyzed to determine trends, patterns, and insights. This includes Natural Language Processing (NLP), sentiment analysis, and topic modelling. The system frequently employs LLMs to improve these operations.

Analytics & Visualization Layer: Extracted insights are grouped and presented for the user interface, allowing them to view trends, summaries, and personalized suggestions.

LLM Integration: **DataTails** LLM services provide advanced language-based analytics and allow users to initiate natural language inquiries.

Figure 3.4 displays the data flow across the DataTails system's major components. The processes are explained below:

Data Ingestion:

- **Extract:** Reddit APIs are utilized to obtain data such post details, comments, and metadata.
- **Transform:** Extracted data is processed, cleansed, and deduplicated before storage and analysis.
- **Load:** The updated data is then stored in a database, such as MongoDB, or in a structured format for future processing.

Data Processing & Storage:

- **Data Cleaning:** This stage removes irrelevant, duplicate, and noisy entries. Missing values are addressed, and data standardization is performed to standardize the material.
- **Feature Extraction:** NLP techniques are used to extract key attributes such as keywords, topics, and sentiment scores.
- **Knowledge Graph Creation:** Structured data is used to generate associations between posts, comments, users, and subtopics.
- **Storage in Database:** Once cleaned and processed, data is saved in a relational or graph database for easy querying and access.

Analysis & Insights Generation:

- **Topic Modeling:** Topic modeling methods, such as **LDA** (Latent Dirichlet Allocation), are used to evaluate posts and comments to discover significant discussion themes.
- **Sentiment Analysis:** Sentiment scores are used to assess the overall sentiment conveyed in posts and comments.
- **Trend Analysis:** Historical trend analysis tracks the popularity of themes throughout time.
- **LLM-Enhanced Insights:** LLMs provide advanced language-based analysis, contextual understanding, and natural language responses to user questions.

Analytics & Visualization Layer:

- **Query Interface:** Users interact with the system using a natural language query interface. **LLM** services assist in interpreting search results and retrieving relevant information.
- **Chart Recommendations:** Based on data properties, the system provides relevant visualizations, such as bar charts, line graphs, and heatmaps, to effectively portray insights.
- **Visualization:** An interactive dashboard displays insights, including trends, sentiment distributions, and other analytical data, for simple study.
- **Report Generation:** The system generates reports based on user-specified parameters, highlighting specific topics or subreddits.

3.2.5 System-Level Sequence Diagram

The **System-Level Sequence Diagram** depicted in Figure 3.7 demonstrates the interaction between the various components of the **DataTails** system. It provides a comprehensive overview of the steps involved in processing a user's query, transforming data, and generating the relevant visualizations for the user.

3.2.5.1 Explanation of the Sequence Flow

1. **Submit Query:** The sequence begins when the **User** submits a query via the **React.js UI**. This marks the initiation of data flow within the **DataTails** system, allowing the system to process the user input.
2. **Backend Processing:** The **React.js UI** passes the submitted query to the **Backend (Flask API)**, which processes the query. The backend system is responsible for orchestrating the subsequent data operations and interaction with other system components.
3. **Web Scraping Initiated by Cron Job:** The **Web Scraper (PRAW API)** is triggered by a scheduled **Cron Job** to extract data from Reddit. The scraped data is stored as unstructured data in **MongoDB**.
4. **Data Storage and Transformation:** After the data has been scraped, it is stored as **Unstructured Data** in **MongoDB**. Subsequently, the **Batch Processing** component retrieves this data and stores it in a structured format for further use.
5. **Knowledge Graph Generation:** The structured data is transferred to the **Knowledge Graph**, where the relationships between different entities are established us-

ing **Ontologies**. This step is essential for transforming raw data into meaningful information.

6. **Contextual Response Generation Using RAG Based LLM:** The **Knowledge Graph** data is fed into the **RAG-Based LLM** to generate a contextual response. This allows the system to generate accurate and relevant results based on the user's query and the structured knowledge available.
7. **Visualization Recommendation:** The **RAG-Based LLM** passes the contextual information to the **Visualization Recommendation System**, which recommends the most suitable visual representation of the data. The recommendation considers the user's subscription type and the nature of the data.
8. **Provide Visualization Options to the User:** Finally, the recommended visualization is provided back to the **User** through the **React.js UI**. The **System** actor is responsible for ensuring the correct visual options are sent based on user type (basic or premium), allowing for either simple or advanced visualization options.

3.2.5.2 Key Components of the Sequence Diagram

- **User Interaction:** The diagram begins with a user-initiated interaction, highlighting the importance of user input in driving the entire data processing pipeline. The **React.js UI** serves as the user's interface to the system.
- **Data Scraping and Storage:** The **Web Scraper (PRAW API)** collects unstructured data from Reddit, which is then stored in **MongoDB**. The use of **Batch Processing** transforms this unstructured data into a structured format that can be further analyzed.
- **User-Specific Visualization:** Based on the user's subscription type, either basic or premium, different types of visualizations are offered to the user. This allows for tailored user experiences, enhancing the system's accessibility and relevance.

The **System-Level Sequence Diagram** depicts how unstructured data, batch processing, knowledge graphs, and an LLM-based response mechanism work together to transform a user's inquiry into meaningful visual insights.

3.2.6 State Transition Diagram

The **State Transition Diagram** in Figure 3.8 demonstrates how a user's query travels through several states via the DataTails system. This graphic depicts how various actions

and user subscription types impact the path taken by the query, resulting in either basic or complex visualizations.

3.2.6.1 Explanation of the State Flow

1. **User Submits a Query:** The process begins when a **User** sends a query to the system. This initial state controls the future flow of actions within the **DataTails** system.
2. **User Subscription Type:** After submitting the query, the system checks the User Subscription Type. This is an important branching point that distinguishes between **premium** and **non-premium** users, deciding their level of access and the features accessible.
3. **Access Premium Features / Access Non-premium Features:** Depending on the user's subscription type:
 - **Premium User:** The user obtains access to **Premium Features** and can select from **all topics** accessible.
 - **Non-premium User:** The user gains access to **Non-premium Features**, restricting them to choose from **up to 3 topics**.
4. **Topic Modelling + Knowledge Graph:** For both premium and non-premium users, the next step involves processing the selected topics using **Topic Modelling and Knowledge Graph** generation. This step helps establish the relationships and structure for the unstructured data relevant to the user's query.
5. **Graph RAG Chatbot:** After the knowledge graph is generated, the **Graph RAG Chatbot** utilizes the structured data to provide a contextual response. This ensures that the user's query is addressed comprehensively, utilizing relevant data points.
6. **Visualization Recommendation System:** Based on the contextual response generated by the chatbot, the system then passes the information to the **Visualization Recommendation System**. This system determines the best way to visually represent the insights gained.
7. **Advanced Visualization (Premium User) / Basic Visualization (Non-premium User):** The visualization options vary based on the user's subscription:
 - **Premium Users** receive **Advanced Visualization** options that offer more detailed and interactive insights.
 - **Non-premium Users** receive **Basic Visualization** options, which are simpler and limited compared to those available to premium users.

8. **End State:** The process concludes with the delivery of the visualization results to the user. The red circled state represents the **End State**, where the query journey is completed, and the appropriate results are displayed to the user.

3.2.6.2 Key Components of the State Transition Diagram

- **Branching Based on Subscription Type:** The diagram clearly showcases how user subscription status affects the features and visualizations available. Premium users have access to advanced features, while non-premium users are restricted to basic visualizations.
- **Topic Modelling and Knowledge Graph Usage:** Regardless of user type, topic modelling and knowledge graph generation are employed to transform the selected data into meaningful insights.
- **Visualization Differentiation:** The diagram highlights the distinction in visualization output based on user subscription, emphasizing the tailored approach of the **DataTails** system to cater to different user needs.

The **State Transition Diagram** illustrates how user queries are processed and how the system replies differently dependent on subscription type, resulting in a personalized valuable user experience.

3.3 Data Design

The system's data design revolves around grouping the core entities needed for optimal query processing, visualization generation, and knowledge management. This section shows how the system's information domain is converted into data structures, as well as the relationships between entities. The Reddit data processing platform relies heavily on **Post** and **Comment**, two significant entities.

Entity Descriptions:

- **Post:** Represents the primary content shared on the platform. The main characteristics of a **Post** are:
 - `_id`: Each post has a unique identifier.
 - `type`: The type of post.
 - `postTitle`: Title of the post.
 - `postDesc`: Description or content of the post.

- `postTime`: Timestamp indicating when the post was created.
 - `authorName`: The username of the post creator.
 - `noOfUpvotes`: Number of upvotes received.
 - `isNSFW`: Boolean indicating if the post is marked as Not Safe For Work (NSFW).
 - `noOfComments`: The count of comments the post has.
 - `imageUrl`, `postUrl`, `subReddit`: URL for images, the post, and the subreddit to which the post belongs.
 - `comments`: An array of **Comment** objects associated with the post.
- **Comment**: Represents comments made on a post. The key attributes of a **Comment** include:
 - `_id`: Unique identifier for each comment.
 - `content`: The content of the comment.
 - `author`: Username of the commenter.
 - `time`: Timestamp indicating when the comment was made.

Relationship Explanation:

- The relationship between **Post** and **Comment** is represented by `has_comments`, which shows that each **Post** can have multiple comments (1..N). This is indicated by the cardinality labels in the diagram, where the **Post** can be linked to many **Comment** entities.
- Each **Comment** must belong to a **Post**, which is why the relationship is from **Post** to **Comment**.

Figure 3.9 provides a graphical representation of the JSON schema utilized in MongoDB to manage and store the relationships between posts and their respective comments. This structure facilitates efficient querying and aggregation for visualization and further processing.

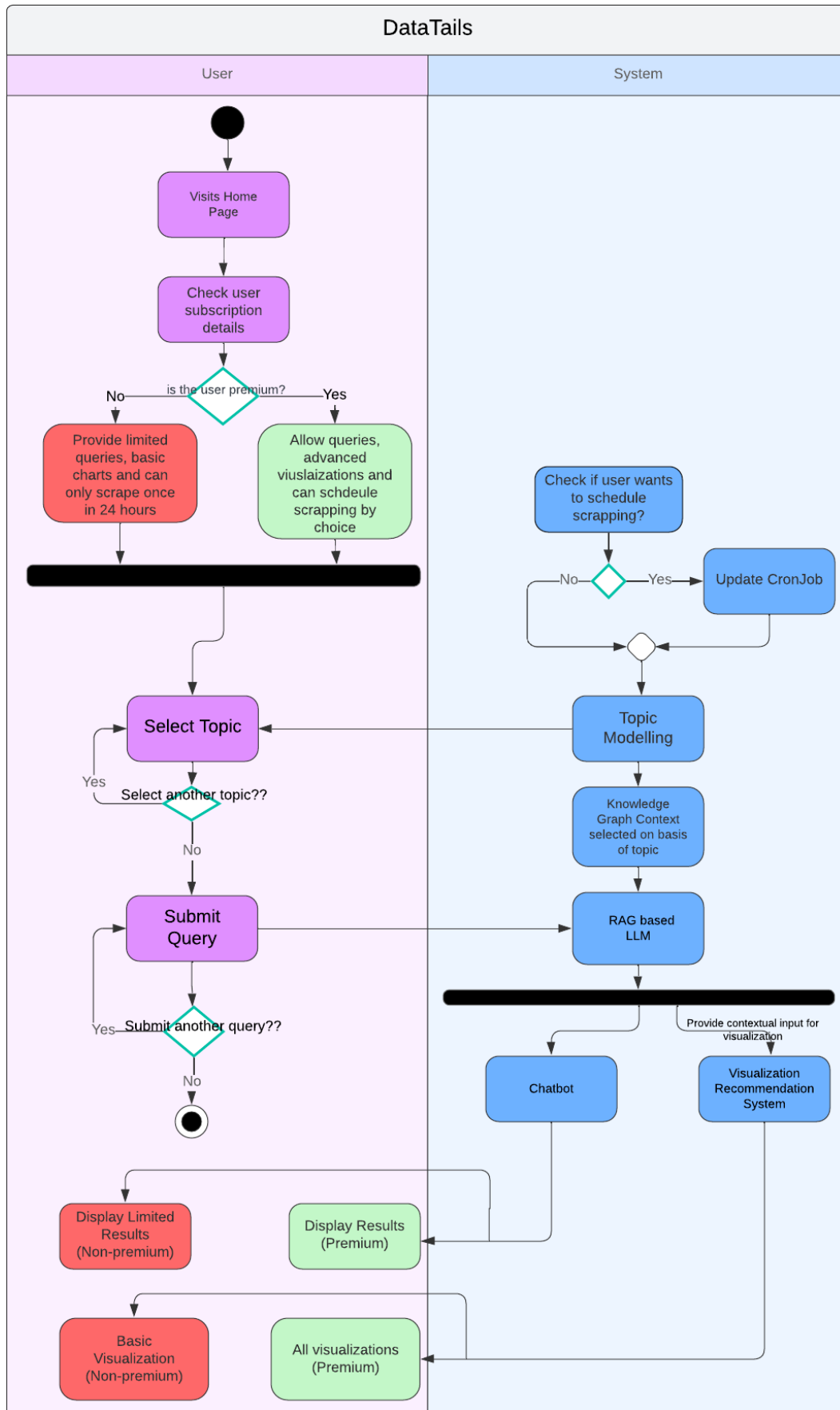


Figure 3.3: Activity Diagram for DataTails

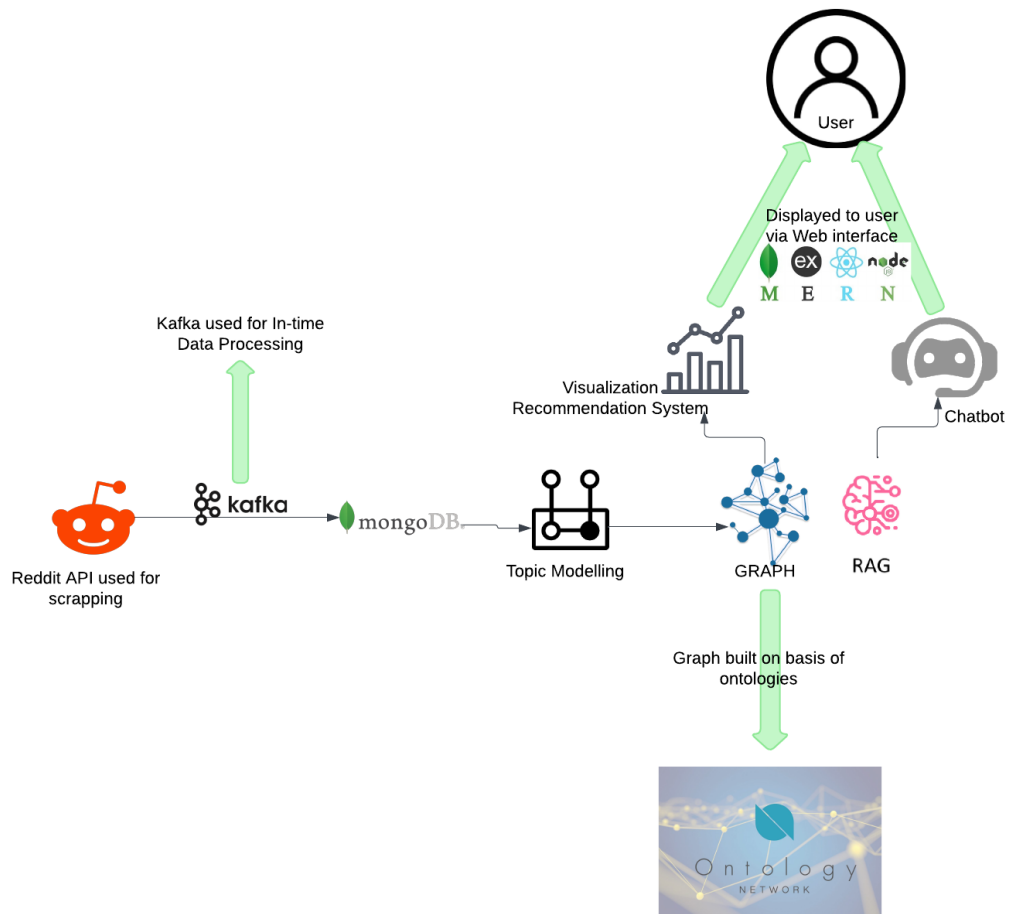


Figure 3.4: Level 2 Data Flow Diagram for DataTails

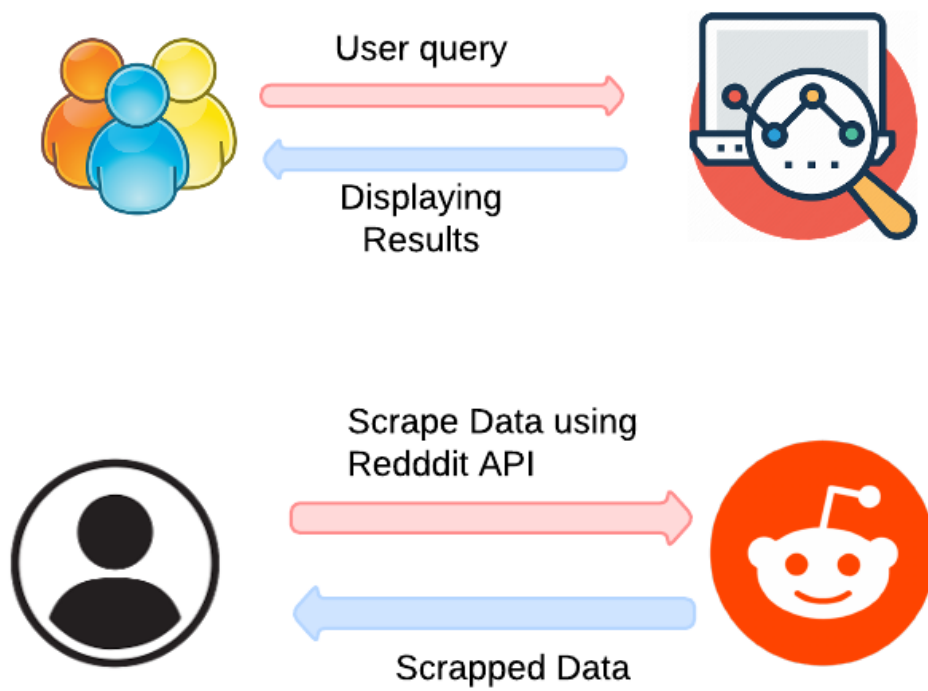


Figure 3.5: Level 0 DataFlow Diagram

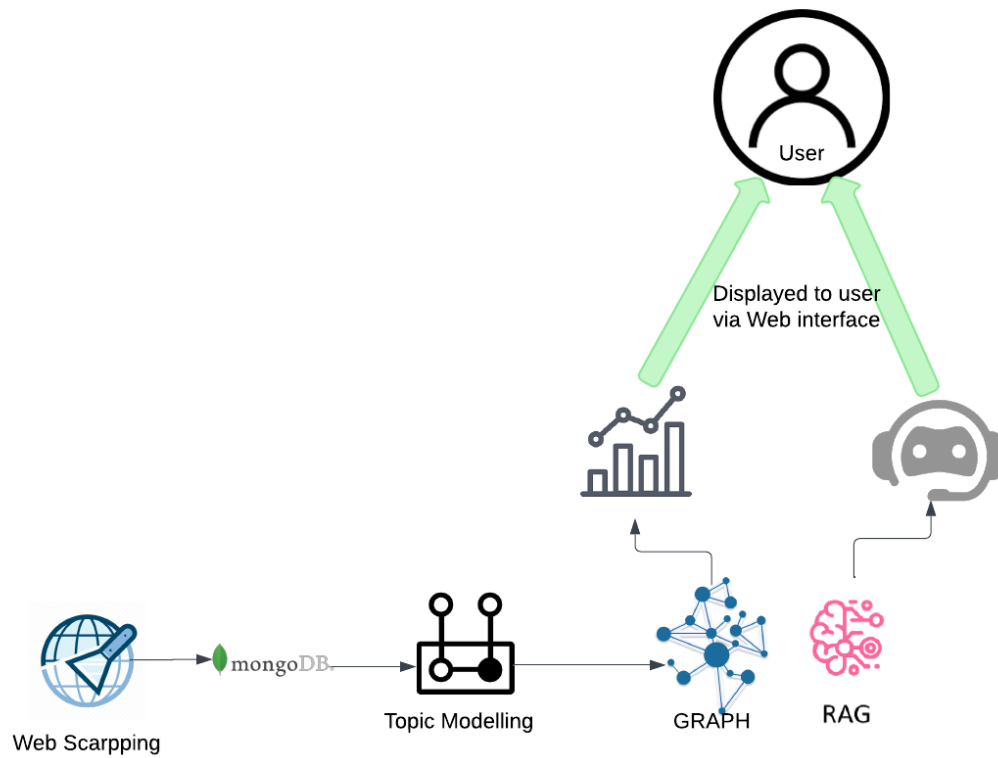


Figure 3.6: Level 1 DataFlow Diagram

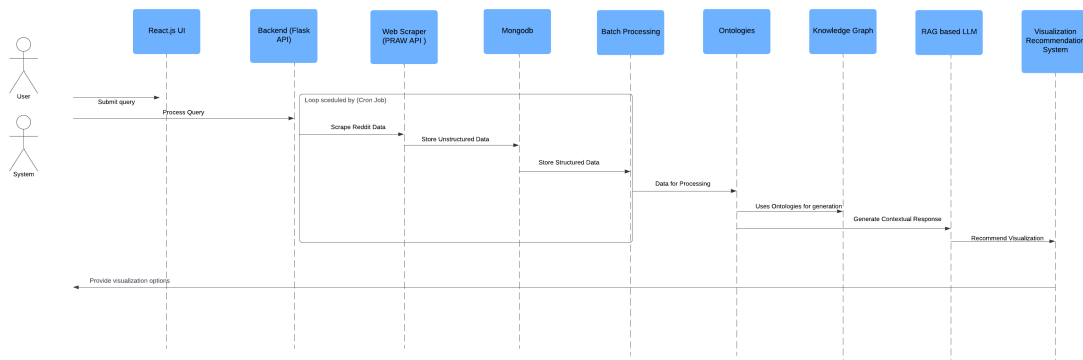


Figure 3.7: System-Level Sequence Diagram for DataTails

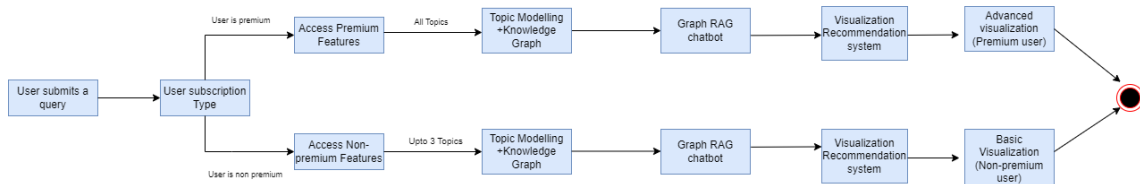


Figure 3.8: State Transition Diagram for DataTails

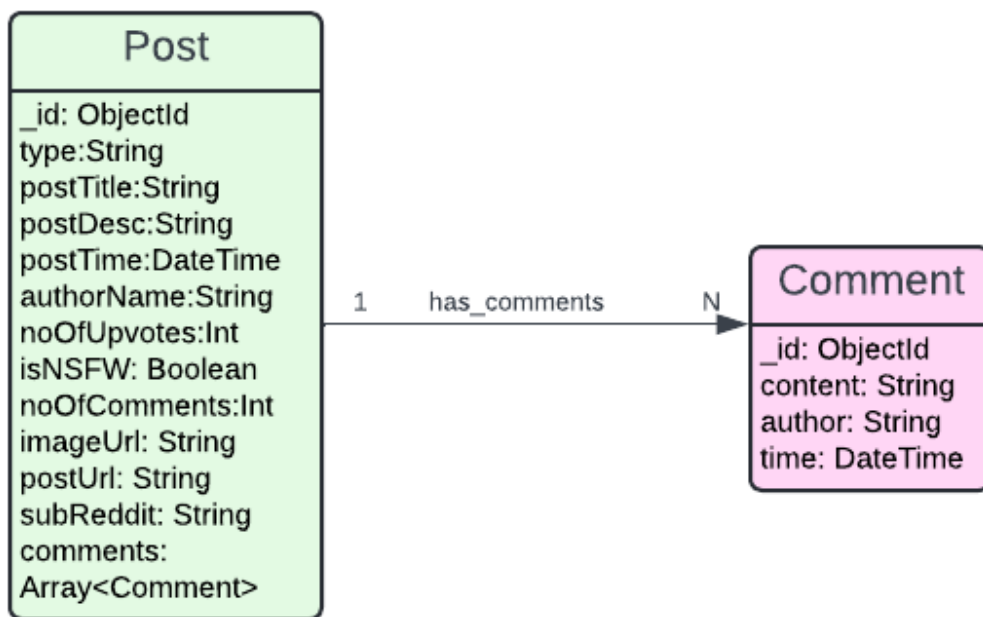


Figure 3.9: MongoDB JSON Schema Diagram for Posts and Comments

Bibliography

- [1] John Smith and Jane Doe. Data scraping and information extraction: A survey of techniques and tools. *Data Mining and Knowledge Discovery*, 37:1–25, 2023.
- [2] Piyush Somani, Sanjay Kumar Dubey, and Amit Agarwal. A survey on data scraping techniques. *Computer Science & Information Technology (CS & IT)*, 7(6):97–102, 2017.